UNIVERSIDAD MARIANO GALVEZ SEDE: CHIMALTENANGO, COLEGIO MESOAMERICANO FACULTA DE INGENIERIA EN SISTEMAS CURSO: BASE DE DATOS I ING. HECTOR JAHAZIEL LUNA IRIARTE

TEMA	**Plataforma de Apuesta**
FECHA	**12 de octubre de 2026**

CARNÉ	NOMBRE COMPLETO	FIRMA
1990-24-12952	**Oscar Eduardo Girón Morales**	
1990-24-25116	**José Nehemías Pichiyá Apén**	
1990-24-13391	**Allan Anthony Ajuchán Popol**	
1990-24-14979	**Diego Julio Alberto Mox Sujuy**	

ÍNDICE

2. OBJETIVOS 3
2.1 OBJETIVO GENERAL 3
2.2 OBJETIVOS ESPECÍFICOS 3
3. ¿CÓMO FUNCIONA UNA APUESTA EN LÍNEA? 4
4. LA PLATAFORMA DE APUESTAS 4
5. PYTHON Y DJANGO 5
5.1 PYTHON 5
5.2 ORIGEN DE DJANGO 5
5.3 ¿QUÉ ES DJANGO? 5
5.4 PATRÓN MTV 6
5.5 ESTRUCTURA DE UN PROYECTO DJANGO 6
5.6 LAS SEIS APPS DE LA PLATAFORMA 6
6. DJANGO EN EL FRONT-END 7
7. RESUMEN DE TECNOLOGÍAS DEL PROYECTO 7
8. BASE DE DATOS 8
8.1 CONCEPTOS DE BASE DE DATOS RELACIONAL QUE USA EL PROYECTO 8
8.2 SQL SERVER Y LA CONEXIÓN CON DJANGO 9
8.3 EL ORM EN ACCIÓN: PYTHON EN LUGAR DE SQL 9
8.4 MODELO DE DATOS: 38 TABLAS 10
9. ARQUITECTURA: RECORRIDO DE UNA PETICIÓN 10
10. EJEMPLO: UNA APUESTA DE PRINCIPIO A FIN 11
11. SEGURIDAD DEL SISTEMA 11
12. LÓGICA DE NEGOCIO Y TRANSACCIONES ATÓMICAS 12
13. PRUEBAS REALIZADAS Y DESPLIEGUE 12
13.1 PRUEBAS DURANTE EL DESARROLLO 12
13.2 PUESTA EN MARCHA 13
14. CONCLUSIONES 13

1. INTRODUCCIÓN

Casi todo lo que hacemos hoy en línea, desde pagar un servicio hasta ver un partido, depende de datos que alguien tiene que guardar y cuidar bien. Las apuestas deportivas son un caso muy claro: el sistema necesita saber quién es cada persona, cuánto dinero tiene, qué cuota se le ofreció justo en el momento de apostar y cuánto corresponde pagar cuando termina el partido.

En el curso de Bases de Datos I nos tocó construir una plataforma con ese desafío. Se hizo con **Python y Django**, y con una base de datos relacional de **38 tablas** en **SQL Server**. Aunque el corazón del proyecto es la base de datos, la aplicación que la rodea es lo que la hace usable: formularios, paneles, reglas de negocio y medidas de seguridad.

Esta investigación se centra en la parte tecnológica. Revisa con qué herramientas se armó el sistema, qué es Django y cómo organiza un proyecto, cómo se guardan y relacionan los datos, y de qué manera se protege el dinero de los usuarios. Para ello se tomaron como base los manuales técnicos y de usuario del proyecto y la documentación del back-end elaborada por el equipo, complementados con conceptos generales de Django y de bases de datos relacionales.

2. OBJETIVOS

2.1 OBJETIVO GENERAL

Comprender la tecnología con la que se construyó la plataforma de apuestas deportivas y reconocer por qué Django, junto con una base de datos relacional en SQL Server, es una combinación adecuada para un sistema que maneja dinero y necesita datos confiables.

2.2 OBJETIVOS ESPECÍFICOS

- Entender qué es Django, de dónde viene y cómo ordena un proyecto con el patrón MTV.
- Revisar cómo está organizada la base de datos y cómo se comunica con la aplicación.
- Seguir el camino de una petición completa, desde el clic del usuario hasta el dato guardado.
- Identificar las medidas que protegen el dinero y la información de los usuarios, y qué aspectos podrían mejorar.

3. ¿CÓMO FUNCIONA UNA APUESTA EN LÍNEA?

Antes de hablar de tecnología conviene entender el negocio que el sistema representa. La plataforma simula el motor de datos de una casa de apuestas deportivas: registra eventos de distintas ligas, ofrece mercados de apuesta, recibe apuestas a través de una billetera virtual y liquida los boletos cuando se conoce el resultado oficial. Estos son los términos que aparecen en todo el sistema:

Término	Qué significa
Evento	Un partido o competencia programada entre dos equipos o atletas
Mercado	Tipo de apuesta ofrecida sobre un evento, por ejemplo "Ganador del partido"
Opción y cuota	Cada alternativa de un mercado (gana local, empate...) con su cuota. Ganancia potencial = monto x cuota
Boleto simple	Apuesta con una sola selección
Boleto combinado	Apuesta con varias selecciones; las cuotas se multiplican y todas deben acertar
Liquidación	Proceso que registra el resultado oficial y define qué boletos ganaron o perdieron

Por ejemplo, una apuesta de Q50 a cuota 1.85 deja una ganancia potencial de Q92.50.

4. LA PLATAFORMA DE APUESTAS

El sistema distingue tres tipos de acceso. El **visitante** puede consultar la cartelera de eventos de hoy y los resultados de ayer, pero no apostar. El **usuario registrado** cuenta con una billetera virtual, deposita y retira fondos, arma boletos simples o combinados y revisa su historial. El **personal administrativo** ingresa por un formulario independiente y, según su rol (operador de eventos, cajero, auditor o súper administrador), gestiona eventos, cuotas, liquidaciones y retiros.

Entre sus funciones principales están:

- Registro con validación de mayoría de edad y una pregunta de seguridad para recuperar la contraseña.
- Eventos con mercados y opciones de apuesta; toda cuota debe ser mayor a 1.00.
- Boletos simples o combinados, con la cuota congelada en el momento de apostar.
- Liquidación automática: con el resultado oficial se paga o se cierra cada boleto.
- Billetera virtual con depósitos, retiros y un historial de cada movimiento de saldo.
- Todas las cantidades de dinero se manejan en quetzales (Q).

5. PYTHON Y DJANGO

5.1 PYTHON

Python es un lenguaje de sintaxis clara y legible, muy usado en desarrollo web, análisis de datos y automatización. Su gran ecosistema de librerías acelera el desarrollo, y por eso es la base sobre la que se construyó el proyecto.

5.2 ORIGEN DE DJANGO

Django nació alrededor de 2003 en la redacción de un periódico de Kansas (Lawrence Journal-World), donde necesitaban publicar sitios web con mucha rapidez. Se liberó como software de código abierto en 2005 y su nombre rinde homenaje al guitarrista Django Reinhardt. Hoy lo mantiene la Django Software Foundation y se utiliza en sitios de gran tamaño como Instagram, Pinterest o Mozilla. Cuenta además con versiones LTS (de soporte a largo plazo); la versión 4.2, utilizada en este proyecto, es una de ellas.

5.3 ¿QUÉ ES DJANGO?

Django es un **framework web** de código abierto escrito en Python. Un framework no es solo un conjunto de herramientas: ya trae una estructura definida y el desarrollador programa dentro de ella. Su lema es "baterías incluidas" y sigue el principio **DRY** (no te repitas). Entre sus componentes principales están:

ORM: permite usar la base de datos con clases de Python.

Plantillas: generan el HTML dinámico de cada pantalla.

URLs: conectan cada dirección con una vista.

Administración: panel generado a partir de los modelos.

Seguridad: protección contra CSRF, XSS y clickjacking, hash de contraseñas y consultas parametrizadas.

5.4 PATRÓN MTV

Django sigue el patrón MTV (Modelo, Plantilla, Vista), muy parecido al MVC clásico:

Django	Equivale a	Función
Modelo	Modelo	Define las tablas y los datos
Vista	Controlador	Lógica: decide qué responder
Plantilla	Vista	El HTML que ve el usuario

Django es en esencia un framework de back-end. Su aporte al front-end es el sistema de plantillas, que genera el HTML desde el servidor (server-side rendering).

5.5 ESTRUCTURA DE UN PROYECTO DJANGO

Un proyecto se divide en **apps**: módulos independientes, cada uno con una responsabilidad. Dentro de cada app, los archivos tienen un rol fijo:

Archivo o carpeta	Para qué sirve
models.py	Define las tablas de la app como clases de Python
views.py	Contiene la lógica: recibe la petición y decide la respuesta
urls.py	Relaciona cada dirección web con una vista
forms.py	Formularios y sus reglas de validación
templates/	Plantillas HTML con las pantallas
migrations/	Historial de cambios que se aplican a la base de datos
settings.py	Configuración global: base de datos, middlewares, sesiones, correo

5.6 LAS SEIS APPS DE LA PLATAFORMA

App	Responsabilidad
catalogos	Datos de referencia: países, departamentos, municipios, estados, deportes, ligas, equipos, tipos de mercado y métodos de pago
cuentas	Credenciales, usuarios, administradores, recuperación de contraseña y bitácoras de acceso

App	Responsabilidad
billetera	Saldo del usuario, transacciones, cuentas bancarias, solicitudes de retiro e historial de saldo
apuestas	Eventos, resultados, mercados, opciones con cuota, boletos y sus historiales
auditoria	Bitácora de seguridad y el middleware de seguridad perimetral
panel_publico	Vista pública con los eventos de hoy y los resultados de ayer; no tiene modelos

6. DJANGO EN EL FRONT-END

Las pantallas del sistema son **plantillas de Django** que heredan de un archivo base (base.html), el cual arma el encabezado con un menú distinto para visitantes, usuarios y administradores. Cada formulario recorre sus campos, muestra los errores de validación e incluye el token CSRF. El estilo es un **CSS propio** (estilos.css), con diseño responsivo basado en CSS Grid y Flexbox, que se adapta a computadora, tableta y teléfono. Además, un context processor propio (sesion_actual) deja los datos de la sesión activa disponibles en todas las plantillas.

El proyecto usa **JavaScript nativo**, sin librerías externas, en dos puntos:

Armado del boleto: al elegir cuotas se suman en el resumen "Mi boleto"; se calcula la cuota total y la ganancia potencial estimada y se envían los identificadores al servidor.

Municipios en cascada: al elegir un departamento, una petición fetch a /cuentas/api/municipios/ llena la lista sin recargar la página.

7. RESUMEN DE TECNOLOGÍAS DEL PROYECTO

Tecnología	Uso en el proyecto
Python + Django 4.2 (LTS)	Framework web: rutas, vistas, plantillas, sesiones y panel de administración
Django ORM	Define las tablas como clases Python y consulta la base sin escribir SQL a mano
SQL Server	Motor de base de datos relacional (SQL Server 2025 en desarrollo)
mssql-django + pyodbc	Conectan Django con SQL Server

Tecnología	Uso en el proyecto
ODBC Driver 18 for SQL Server	Controlador de Microsoft que abre la conexión desde Python
python-decouple	Lee credenciales y configuración desde variables de entorno (.env)
HTML5, CSS y JavaScript nativo	Interfaz responsiva (CSS Grid y Flexbox), sin librerías de terceros
Hash de Django (PBKDF2)	Almacena contraseñas y respuestas de seguridad de forma cifrada

8. BASE DE DATOS

8.1 CONCEPTOS DE BASE DE DATOS RELACIONAL QUE USA EL PROYECTO

Concepto	Qué es	Dónde se ve en la plataforma
Llave primaria (PK)	Identifica de forma única cada fila	id_usuario, id_boleto, id_evento...
Llave foránea (FK)	Enlaza una fila con otra tabla	Un boleto apunta a una billetera; una opción, a un mercado
Integridad referencial	Impide dejar datos huérfanos o borrar lo que otros usan	on_delete=PROTECT en catálogos y registros financieros
Restricción CHECK	Obliga a que un valor cumpla una condición	Saldos mayores o iguales a 0, cuota mayor a 1.00
Unicidad	Evita duplicados	Una opción no se repite dentro de un mismo boleto
Índices	Aceleran las búsquedas frecuentes	Fecha de inicio del evento, IP y fecha en la bitácora
Transacción (ACID)	Un grupo de cambios se aplica completo o no se aplica	Crear un boleto o liquidar un evento

8.2 SQL SERVER Y LA CONEXIÓN CON DJANGO

El proyecto usa una única base de datos, **bd_apuestas_deportivas**, creada con el script sql/bd_apuestas_deportivas.sql. Django se conecta mediante el motor **mssql** y el controlador **ODBC Driver 18**. Las credenciales no están escritas en el código: se leen del archivo.env con python-decouple. Ejemplo ilustrativo (simplificado) de la configuración en settings.py:

```
DATABASES = {"default": {
    "ENGINE": "mssql",
    "NAME": config("DB_NAME"),
    "USER": config("DB_USER"),
    "PASSWORD": config("DB_PASSWORD"),
    "HOST": config("DB_HOST"),
    "PORT": config("DB_PORT"),
    "OPTIONS": {"driver": "ODBC Driver 18 for SQL Server",
                "extra_params": "TrustServerCertificate=yes"},
}}
```

8.3 EL ORM EN ACCIÓN: PYTHON EN LUGAR DE SQL

Cada clase de models.py es una tabla. El nombre real se fija con **db_table** y las llaves foráneas usan **db_column** para coincidir con el script SQL. Ejemplo ilustrativo (simplificado):

```
class Pais(models.Model):
    nombre_pais = models.CharField(max_length=100)
    codigo_iso = models.CharField(max_length=5)

    class Meta:
        db_table = "tbl_pais"
```

Para consultar los países con código GT se escribe:

```
Pais.objects.filter(codigo_iso="GT")
```

Django lo traduce a una sentencia equivalente a SELECT * FROM tbl_pais WHERE codigo_iso = 'GT'. Los valores viajan parametrizados, lo que ya protege contra la inyección SQL.

8.4 MODELO DE DATOS: 38 TABLAS

La base tiene 38 tablas con prefijo **tbl**_ y nombres en español, repartidas por dominio:

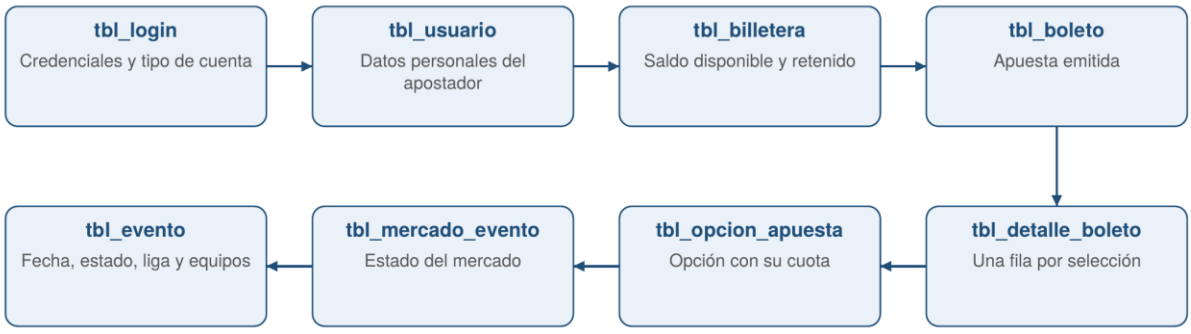
Dominio	Tablas	Ejemplos
Catálogos	9	tbl_pais, tbl_deporte, tbl_liga, tbl_metodo_pago
Cuentas y acceso	12	tbl_login, tbl_usuario, tbl_recuperacion_password
Billetera	5	tbl_billetera, tbl_transaccion, tbl_solicitud_retiro
Apuestas	11	tbl_evento, tbl_opcion_apuesta, tbl_boleto
Auditoría	1	tbl_bitacora_seguridad

tbl_login es la raíz de las cuentas: un login puede ser usuario, administrador o super administrador.

tbl_estado es un catálogo genérico que reutilizan usuarios, administradores, billeteras y deportes.

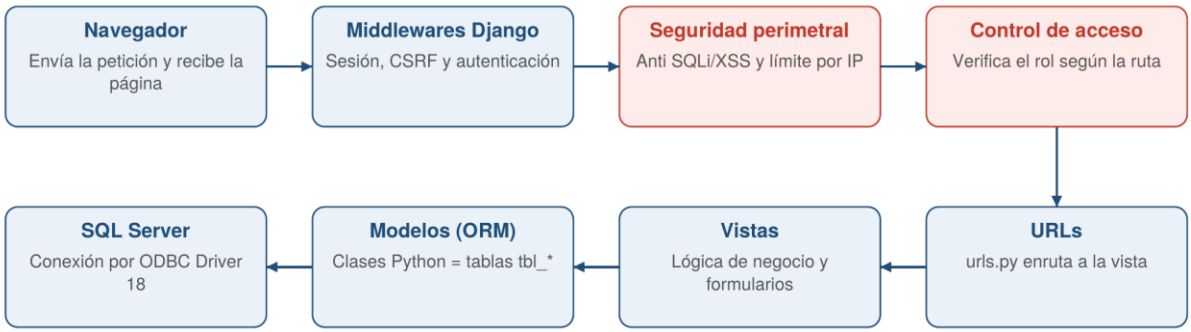
Los historiales (cuotas, estados de boleto, saldos, eventos) dejan trazabilidad de cada cambio.

La cadena de relaciones va de la cuenta al partido: un login tiene un usuario, el usuario una billetera y la billetera varios boletos. Cada boleto se desglosa en detalles que apuntan a opciones de apuesta, y cada opción pertenece a un mercado de un evento deportivo.



9. ARQUITECTURA: RECORRIDO DE UNA PETICIÓN

La petición entra por los middlewares, la ruta la envía a una vista, la vista usa los modelos para leer o escribir en SQL Server y una plantilla HTML arma la página de respuesta. Los dos middlewares propios del proyecto (seguridad perimetral y control de acceso, en rojo en la figura) se ejecutan antes de que la petición llegue a cualquier vista.



10. EJEMPLO: UNA APUESTA DE PRINCIPIO A FIN

Momento	Qué ocurre en el sistema	Disponible	Retenido
Antes de apostar	El usuario tiene saldo en su billetera	Q600.00	Q0.00
Confirma el boleto	Apuesta Q50 a cuota 1.85. El monto pasa a retenido, el boleto queda Pendiente y la ganancia potencial es Q92.50	Q550.00	Q50.00
Se liquida el evento (gana)	Se libera el monto retenido y se acredita la ganancia potencial; el boleto pasa a Ganado	Q642.50	Q0.00

Si la selección hubiera perdido, el boleto pasaría a Perdido y solo se liberaría el monto retenido, sin acreditar ganancia. En un boleto combinado con cuotas 2.00 y 1.50, la cuota total sería 3.00; basta que una selección falle para perder. Este mismo escenario (Q600, Q50, cuota 1.85, saldo final Q642.50) fue una de las pruebas realizadas durante el desarrollo.

11. SEGURIDAD DEL SISTEMA

Mecanismo	Cómo funciona
Hash de contraseñas	Se guardan con make_password (PBKDF2); también las respuestas de seguridad. Nunca en texto plano
Filtro anti inyección	Un middleware revisa GET y POST buscando patrones de SQL injection y scripts; responde HTTP 403
Límite de peticiones	Máximo 60 por minuto por IP; al excederlo responde HTTP 429 (mitigación de DDoS)

Mecanismo	Cómo funciona
Bloqueo de cuenta	5 intentos fallidos seguidos bloquean la cuenta durante 15 minutos
Control de acceso por rol	Tres capas: middleware por ruta, decorador en cada vista y verificación del rol administrativo
Sesión y cookies	Sesión de 4 horas, cookies HttpOnly, protección CSRF y X-Frame-Options en DENY
Recuperación de contraseña	Correo y pregunta de seguridad, luego un token UUID de un solo uso con 30 minutos de vigencia
Validación de montos	Montos mayores a 0 y cuotas mayores a 1.00, tanto en formularios como con CHECK en la base

12. LÓGICA DE NEGOCIO Y TRANSACCIONES ATÓMICAS

Proceso	Pasos clave
Crear boleto	Valida el monto y las opciones, calcula cuota total y ganancia potencial, bloquea la billetera, mueve el dinero de disponible a retenido y guarda la cuota del momento
Liquidar evento	Registra el resultado, marca las opciones ganadoras, cierra los mercados y paga o cierra los boletos cuyos eventos ya finalizaron
Gestionar retiros	El cajero aprueba o rechaza; al aprobar se verifica otra vez el saldo, se descuenta y la solicitud pasa a Pagada

Cada operación que toca el saldo corre dentro de **transaction.atomic()**: se completa por entero o no se aplica ningún cambio. Con **select_for_update()** se bloquea la fila de la billetera, para que dos solicitudes simultáneas no dejen un saldo inconsistente.

13. PRUEBAS REALIZADAS Y DESPLIEGUE

13.1 PRUEBAS DURANTE EL DESARROLLO

Carga correcta de las rutas principales.

Inicio de sesión de usuario y de administrador.

Depósito de fondos y creación de boletos.

Liquidación con pago automático (de Q600 a Q642.50).

Bloqueo de SQL injection y registro en la bitácora de seguridad.

Rechazo de montos negativos.

Límite de peticiones (HTTP 429).

13.2 PUESTA EN MARCHA

Instalar Python, SQL Server y ODBC Driver 18.

Crear el entorno virtual e instalar las librerías.

Configurar el archivo .env con las variables DB_*.

Ejecutar el script SQL de la base de datos.

Correr **python manage.py migrate --fake-initial** para crear las tablas propias de Django.

Iniciar con **python manage.py runserver**.

En producción: DEBUG en False, HTTPS, servidor SMTP real y respaldos periódicos de la base.

14. CONCLUSIONES

Django hace que el proyecto sea ordenado: el patrón MTV separa datos, lógica e interfaz, y las seis apps dejan claro quién se encarga de qué.

El ORM conecta el código Python con las 38 tablas de SQL Server sin escribir SQL a mano, y de paso reduce el riesgo de inyección SQL.

Como el sistema maneja dinero, la integridad se cuida en dos niveles: restricciones CHECK en la base y transacciones atómicas en la aplicación.

La seguridad funciona por capas, no depende de una sola medida, y todavía tiene espacio para mejorar con las extensiones propuestas.